

Project 10 - Online Medicine Shop

Course: Web Technologies | **Format:** Project | **Duration:** 1 week

Application Overview

An “Online Medicine Shop” that provides a wide range of medicines at customers’ dire necessity. Two types of users: **ADMIN** and **CUSTOMER**.

- **ADMIN:** administrative capabilities – adds, deletes, modifies medicine information, deletes existing customers, accepts or rejects order lists, sees all purchase request details, sees purchase list history of all customers.
- **CUSTOMER:** registered user – browses medicines, searches with filters (vendor, genre), adds medicines to cart with quantity, increases/decreases quantity, removes items, confirms purchase with address, views invoice, selects payment method, sees successful order list after confirmation.

Team Structure & Student IDs

Student ID	Task	Main Features
23-50003-1	Task 1	User Authentication (Admin/Customer), Profile Management, Home Page, Medicine Browsing by Categories, Search & Filtering (AJAX)
23-50144-1	Task 2	Admin – Medicine Management (full CRUD), Category Management (with liquid/solid segmentation), Delete Customers, View All Purchase Requests, Accept/Reject Orders, View All Customers’ Purchase History
	Task 3	Customer – Add to Cart, Cart Management (increase/decrease quantity, remove), Checkout with Address, Invoice, Payment Method Selection, Order Confirmation

Shared Database Schema (same for all)

Table	Key Columns
users	id, name, email, password_hash, role (admin/customer), profile_picture, address, phone, created_at
categories	id, name (e.g., Aspirin genre, Paracetamol genre), category_type (liquid/solid), created_at
medicines	id, name, category_id, vendor_name, price, availability (stock), description, image_path, created_at
cart	id, user_id, medicine_id, quantity, added_at
orders	id, user_id, total_amount, shipping_address, status (pending/accepted/rejected), payment_method, order_date
order_items	id, order_id, medicine_id, quantity, unit_price
payments	id, order_id, amount, payment_method, transaction_id, payment_date

No code sharing except the schema.

Global Technical Requirements

- PHP MVC (controllers/, models/, views/, config/)
- Passwords hashed with `password_hash()` / verified with `password_verify()`
- PDO/mysqli + **prepared statements** for every query – no string-concatenated SQL
- Server-side validation on every form before any DB write; show inline error messages
- `session_start()` on every page that requires auth; redirect unauthenticated users

- AJAX endpoints return Content-Type: application/json; handle errors client-side
- File uploads go to public/uploads/ (profile pictures, medicine images) – validate MIME type + size server-side
- Do not drop or alter the shared schema tables

Mandatory Grading Criteria (each student must satisfy all 10 within their task)

1. **Basic Web Security** – SQL injection prevention, XSS protection, CSRF awareness, secure password storage.
2. **UI (HTML/CSS)** – Clean, responsive, user-friendly interface.
3. **Feature Completeness** – All assigned requirements work error-free.
4. **DB** – Correct use of shared schema, proper relationships, data integrity.
5. **Auth (Session/Cookie)** – Session management, role-based access, “Remember Me” (optional).
6. **MVC** – Clear separation of business logic, presentation, and request handling.
7. **JS Validation** – Client-side validation on forms (e.g., registration, medicine upload, cart quantity, checkout).
8. **PHP Validation** – Server-side validation on every input before DB write.
9. **Ajax/JSON** – At least one AJAX endpoint returning JSON per student.
10. **Git Contribution** – Feature branches, meaningful commits (≥ 3 per student), merge into main via PR.

Git Flow

- Repository has a main branch (protected, no direct pushes).
- Each student creates a **feature branch** from main named feature/taskX-studentID (e.g., feature/task1-2355415-3).
- Work on the feature branch, commit regularly (at least 3 commits per student).
- After completing the feature, open a **pull request** into main and merge.
- Final main must contain all three feature branches merged with full history.

Task Breakdown

Task 1 – User Authentication (Admin/Customer), Profile Management, Home Page, Medicine Browsing by Categories, Search & Filtering (AJAX)

Student: 23-50003-1

CRUD included: Create (registration), Read/Update (profile).

Requirements:

- **Registration** – form for ADMIN and CUSTOMER. Collect: name, email, password (≥ 8 chars), address, phone. Role dropdown. Validate unique email, hash password. Redirect to login.
- **Login & Remember Me** – login creates `$_SESSION['user_id']`, `$_SESSION['name']`, `$_SESSION['role']`. “Remember Me” cookie + token for auto-login.
- **Profile Page** (session-gated) – both admin and customer can update name, email, address, phone, profile picture, change password (require current). Navigation to profile from home page.
- **Logout & Navbar** – destroy session and cookie. Navbar shows links based on role.
- **Home Page** – shows:
 - Category listing (Aspirin genre, Paracetamol genre, etc.) and logical segmentation (liquid/solid).
 - Medicine cards (name, vendor, price, availability).

- **Medicine Browsing by Categories** – clicking a category shows medicines under that category. Filter by liquid/solid if applicable.
- **Search Box & Filtering (AJAX)** – search by medicine name, with filters: vendor name, genre (category). AJAX GET /api/medicines/search?q=&vendor=&genre= returns JSON; JS dynamically updates medicine list without page reload.
- **JS validation** on registration and profile forms.

Key outputs: users table with admin and customer accounts; session management; home page with category browsing and AJAX search/filter.

Task 2 – Admin: Medicine Management (full CRUD), Category Management, Delete Customers, View All Purchase Requests, Accept/Reject Orders, View All Customers’ Purchase History

Student: 23-50144-1

CRUD included: Full CRUD on medicines and categories + Delete on users (customers) + Read/Update on orders.

Requirements:

- **Admin Gate** – every admin page checks \$_SESSION['role'] === 'admin'; redirect others.
- **Category Management** – create, edit, delete categories. Each category has name (e.g., Aspirin genre) and category_type (liquid/solid) for logical segmentation. Block deletion if medicines exist.
- **Medicine Management** – create, edit, delete medicines.
 - Form: name, category dropdown, vendor name, price (>0), availability (stock), description, image upload (JPEG/PNG ≤2MB to public/uploads/medicines/).
 - Edit pre-fills form. Delete removes image and record (block if in any pending order).

- **Delete Customer** – list all customers (role='customer') with delete button. Cascade remove cart, orders.
- **View All Purchase Requests** – admin page showing all orders (pending/accepted/rejected) with customer name, total amount, shipping address, order date, status.
- **Accept/Reject Orders** – on same page, admin can change order status (accept or reject) via AJAX.
- **View All Customers' Purchase History** – separate admin page showing all completed orders (accepted) with customer details, medicine list, quantities, total.
- **Admin Dashboard** – counts of total medicines, categories, customers, pending orders.
- **PHP & JS validation** on medicine/category forms.
- **AJAX** – for accept/reject order status update.

Key outputs: Admin fully manages medicines and categories, deletes customers, views all purchase requests and history, accepts/rejects orders.

Task 3 – Customer: Add to Cart, Cart Management, Checkout with Address, Invoice, Payment Method Selection, Order Confirmation

Student:

CRUD included: Create, Update, Delete on `cart` (session or DB) + Create on `orders`, `order_items`, `payments`.

Requirements:

- **Customer Gate** – cart and checkout require login; browsing is public (handled by Task 1).
- **Add to Cart** – on each medicine card or detail page, “Add to Cart” button with quantity field (default 1). AJAX POST to `/api/cart/add` with `medicine_id` and `quantity`. Updates DB cart (or session). Returns new cart count and updates navbar cart icon.
- **Cart Page** – list cart items (medicine name, vendor, price, quantity, subtotal).

- Options: increase quantity (+), decrease quantity (-), remove item (AJAX DELETE). Each action calls AJAX endpoint (/api/cart/update, /api/cart/remove) and returns updated totals.
- **Checkout** – from cart page, customer clicks “Proceed to Checkout”.
 - Form: shipping address (pre-filled from profile, editable), confirm.
 - After address submission, show **invoice**: list of medicines, quantities, unit price, total amount.
 - Buttons: “Cancel” (back to cart) and “Confirm Purchase”.
- **Payment Method Selection** – after confirm purchase, show payment methods (Credit Card, bKash, Nagad, Bank Transfer, Cash on Delivery).
 - On payment selection, create order (status='pending'), order_items, clear cart, store payment record.
 - Redirect to **successful order list** page showing order ID, medicines, total, and status “pending admin approval”.
- **JS validation** – cart quantity (positive integer, not exceed stock), address not empty, payment method selected.
- **PHP validation** – medicine existence, stock availability, valid address.

Key outputs: Customers can add to cart, manage quantities, checkout, see invoice, select payment method, and receive order confirmation.